

AI-NATIVE GROCERY MERCHANT

Document 11 — Development Roadmap

Implementation sequence • Dependency-aware • Locked MVP

Roadmap principle: Build the authoritative commerce and safety foundation first, then optimization and agent intelligence, and finally payment, audit and end-to-end hardening.

Document status: LOCKED DEVELOPMENT ROADMAP

Scope rule: This roadmap implements the already-locked MVP. New features require explicit scope review.

1. Roadmap Objective

This document converts Documents 01–10 into an executable implementation sequence. Work is ordered by dependency so payment and agent layers are built on stable commerce and authorization foundations.

Core invariant

The AI reasons. The user chooses. The policy engine authorizes. Razorpay executes. The audit trail records.

Architecture rule

- Use a modular monolith for the MVP.
- Keep catalog, cart, policy, payments and audit server-authoritative.
- Keep the AI behind explicit tool adapters.
- Use deterministic services for money, stock, incentives and authorization.
- Use seeded data for reproducible development and demos.

2. Development Order

Phase	Outcome	Depends on
0	Project foundation	None
1	Commerce core	0
2	Mandate + policy	1
3	Shopping intent	1
4	Research + quality	1, 3
5	Optimization + incentives	1, 2, 3, 4
6	AI agent orchestration	2, 3, 4, 5
7	User selection + fresh quote	2, 5, 6
8	Razorpay Test Mode	2, 7
9	Audit + observability	All
10	E2E/security/demo hardening	All

3. Phase 0 — Project Foundation

Goal

Create a clean, reproducible baseline.

Build

- Repository/module structure.
- Environment configuration.
- Database and migrations.
- Development/test separation.
- Logging and correlation IDs.
- Authentication/session foundation.
- Common API error model.
- Test runner and CI/check commands.
- Razorpay Test Mode configuration without committed secrets.

Exit gate

Clean startup, migrations, health check, automated tests and safe configuration.

4. Phase 1 — Commerce Core

Goal

Establish the authoritative merchant-side commerce model.

Build

- Product/SKU and category entities.
- Price and integer minor-unit money model.
- Stock state.
- Pack-size and quantity attributes.
- Catalog search/filter service.
- Cart and cart-item model.
- Server-side subtotal/discount/payable calculation.
- Application order state model.

Exit gate

Seeded products can be searched, placed into a basket, priced deterministically and persisted.

5. Phase 2 — Mandate & Policy

Goal

Build the independent financial authorization boundary before payment integration.

Build

- Mandate lifecycle.
- Allowed-category check.
- Maximum per-item check.
- Stock check.
- Incentive validity check.
- Final amount calculation.
- Maximum-spend check.
- ALLOW/DENY decision object and reason codes.
- Policy versioning.
- Decision persistence.
- Pre-payment revalidation.

Exit gate

Over-limit baskets are denied and cannot reach Razorpay; feasible baskets are allowed.

6. Phase 3 — Shopping Intent

Goal

Represent natural-language grocery goals as structured requirements.

Build

- Shopping session.
- Shopping intent.
- Requirements and quantities/units.
- Budget.
- Optional quality preference.
- Category/brand constraints.
- Ambiguity and assumption handling.

Exit gate

Inputs such as 'Pasta for 4, budget ■1000' and '6 eggs' become validated requirements without unsupported invention.

7. Phase 4 — Product Research & Quality

Goal

Provide controlled evidence for quality-aware optimization.

Build

- Product evidence model.
- Seeded review/quality fixtures.
- Evidence retrieval.
- Evidence confidence metadata.
- Candidate quality assessment.
- Rules against unsupported allegations.
- Optional research caching.

Exit gate

Quality decisions use stored evidence and do not fabricate claims.

8. Phase 5 — Optimization & Incentives

Goal

Turn requirements into useful, financially bounded baskets.

Build

- Candidate feasibility filtering.
- Requirement fulfillment.
- Quantity optimization.
- Pack-size optimization.
- Quality-aware ranking.
- Deal optimization.
- Smart Voucher Decision Engine.
- Loyalty reward decision.

- Best Value basket.
- Best Quality Within Budget basket.
- Basket comparison/recommendation metadata.
- Deterministic quote generation.

Exit gate

Egg, dual-basket and incentive scenarios pass Document 10 acceptance tests.

9. Phase 6 — AI Agent Orchestration

Goal

Let the LLM reason over approved tools without becoming financial authority.

Build

- Requirement extraction/validation.
- Catalog search tool.
- Evidence tool.
- Optimization tool.
- Incentive tool.
- Basket explanation/comparison.
- Policy evaluation tool.
- Structured denial/recovery loop.

Exit gate

Agent completes planning through approved tools only. It cannot access raw DB/payment controls or override policy.

10. Phase 7 — User Selection & Fresh Quote

Goal

Separate AI recommendation from user preference and authorization.

Build

- Best Value/Best Quality UI.
- Recommendation and trade-off explanation.
- Basket selection endpoint.
- Ownership validation.
- Fresh server-side quote.
- Basket/quote versioning.
- Stale-state detection.

Exit gate

User may choose either basket, but payment requires a fresh quote and current policy ALLOW.

11. Phase 8 — Razorpay Test Mode

Goal

Connect authorized application orders to Razorpay safely.

Build

- Idempotent checkout creation.
- Policy ALLOW gate.
- Razorpay order creation.
- Checkout handoff.
- Server payment verification.
- Webhook endpoint.
- Signature validation.
- Webhook idempotency.
- State reconciliation.
- Amount reconciliation.

Exit gate

Authorized amount matches Razorpay Test Mode order; denied baskets create no provider order.

12. Phase 9 — Audit & Observability

Goal

Make material decisions reconstructable.

Build

- Goal and requirement events.
- Research/quality decisions.
- Optimization decisions.
- Voucher/loyalty decisions.
- Basket generation.
- AI recommendation.
- User selection.
- Fresh quote.
- Policy decision.
- Razorpay/payment state.
- Webhook processing.
- Final order state.

Exit gate

A reviewer can trace one transaction from goal to final payment state.

13. Phase 10 — Hardening & Demo Freeze

Goal

Make the locked MVP reliable and repeatable.

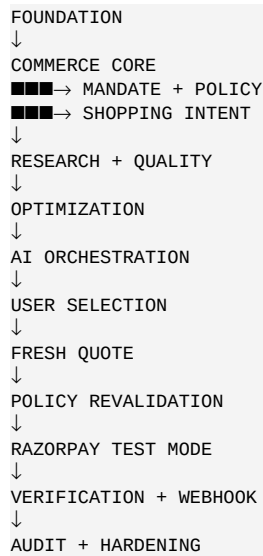
Build

- Run full regression suite.
- Run happy-path E2E.
- Run MAX_SPEND denial/recovery E2E.
- Test stale price/stock/voucher.
- Test client amount manipulation.
- Test checkout without ALLOW.
- Test invalid/duplicate webhooks.
- Test prompt/tool injection.
- Review trust boundaries.
- Remove debug paths/secrets.
- Seed clean demo data.
- Verify audit visibility.
- Freeze build.

Exit gate

Document 10 acceptance checklist passes with no critical defect in the locked MVP path.

14. Dependency Map



15. Parallelizable Work

Workstream	Can overlap with
Frontend shell/UI	Backend development after API shapes are agreed
Seed catalog	Commerce core implementation
AI prompt/tool adapters	Optimization once tool contracts are frozen
Policy test suite	Policy implementation
Webhook handler	Checkout UI once payment contracts are defined
Audit event schemas	All modules after event names are agreed

16. Feature Implementation Loop

1. Define data contract
2. Define deterministic business rules
3. Implement backend service
4. Add unit tests
5. Add API contract
6. Add integration tests
7. Add agent adapter if required
8. Add frontend flow
9. Add audit events
10. Add failure/recovery behavior
11. Add E2E coverage
12. Mark complete only after the phase gate passes

17. Phase Definition of Done

- Implementation matches its specification.
- Authoritative logic is in the correct backend module.
- Normal and failure cases are tested.
- API contracts are validated.
- Material decisions produce audit events.
- No new authority path is introduced.
- No secrets are committed.
- Phase exit gate passes.
- Existing regression tests remain green.

18. MVP Freeze Criteria

- Natural-language grocery goal handling.
- Structured requirements.
- Controlled catalog search.
- Evidence-backed quality research.
- Quantity and pack optimization.
- Deal optimization.
- Smart voucher use/save decision.
- Loyalty reward decision.
- Best Value basket.
- Best Quality basket.
- AI recommendation.
- User final basket selection.
- Deterministic mandate/policy authorization.
- Fresh pre-payment revalidation.
- Razorpay Test Mode transaction.
- Server-side payment verification.
- Webhook handling and idempotency.
- Complete audit trace.
- Graceful policy-denial recovery.

19. Explicitly Deferred Work

- Multi-merchant marketplace.
- Multi-agent architecture.
- Voice commerce.
- Negotiation engine.
- Production/real-money payments.
- Autonomous refunds, payouts or banking actions.
- Full ACP/AP2/UAP certification or protocol implementation.
- Unrestricted external shopping crawler.
- Large-scale recommendation infrastructure.
- Full loyalty platform.
- Generic coupon marketplace.
- Large CRM/ERP/analytics integrations.
- Large-scale concurrency infrastructure.

20. Buildathon Demo Order

Step	Action	Proof
1	Enter grocery goal + budget	Natural-language agent
2	Show extracted requirements	Requirement reasoning
3	Show quality/product evidence	Research
4	Show pack/quantity decisions	Optimization
5	Show Best Value + Best Quality	Dual objective
6	Show voucher/loyalty decision	Smart incentives
7	Show AI recommendation	Reasoning
8	Choose basket	User preference control
9	Show policy ALLOW	Independent authorization
10	Complete Razorpay Test Mode payment	Transactable AI buyer
11	Show audit trail	Explainable money action
12	Run over-budget scenario	Failure + recovery

21. Engineering Principles

- LLM is never the financial authority.
- Frontend totals are never authoritative.
- No Razorpay order before policy ALLOW.
- Always revalidate immediately before payment.
- Product/review text is untrusted input.
- Use deterministic rules for authoritative optimization.
- Use the LLM for interpretation, synthesis, comparison and explanation.
- Fail closed on authorization uncertainty.
- Make retries idempotent.
- Keep demo data deterministic.

- Do not add scope implicitly.

22. Final Roadmap Gate

READY FOR SUBMISSION only when Documents 01–10 are implemented sufficiently for the locked MVP, Document 10's acceptance suite passes, the money path is auditable, and the failure/recovery demonstration works reliably.

The roadmap is complete when the system proves that the AI can perform substantial grocery purchasing intelligence while actual money actions remain bounded by user authority, deterministic policy and Razorpay execution.

23. Related Development Documents

Document	Role
01 — Development Master Spec	Locked product baseline
02 — Functional Requirements	Feature behavior
03 — System Architecture	Component/trust boundaries
04 — AI Agent Specification	Agent/tool rules
05 — Optimization Decision Specification	Optimization logic
06 — Policy & Mandate Specification	Authorization boundary
07 — Data Model Specification	Persistence contracts
08 — API Contract Specification	API contracts
09 — Razorpay Integration Specification	Payment integration
10 — Testing & Acceptance Specification	Verification and release gates